Лабораторная работа № 9.

Интеграция REST API с frontend.

Тема: Учебная административная панель: GET и POST для категорий на Express.js

Цель: Научиться связывать frontend с Express REST API через fetch(), получать JSON через GET, отправлять данные через POST и обновлять DOM.

Дисциплина: Разработка серверных приложений и REST API с использованием Express.js

1. Общие сведения

Лабораторная работа №9. Интеграция REST API с клиентским приложением. Продолжительность — 4 академических часа (180 минут).

Разрабатывается учебная административная панель для работы с данными через REST API. Это не интернет-магазин: не нужны покупатели, корзина, оплата и оформление покупок. Главная цель — связать frontend и backend через HTTP-запросы GET и POST.

2. Тема и цель

Тема: получение и создание категорий из браузера с использованием Fetch API и Express.js.

Цель: самостоятельно реализовать два REST-маршрута для категорий, получать JSON, отправлять JSON из HTML-формы, отображать данные в DOM и обрабатывать ошибки.

3. Оборудование и исходные данные

- ПК с Node.js и npm; VS Code и браузер.
- Исходный проект shop-admin, предоставленный преподавателем, с готовой вкладкой «Товары».
- Express и ES Modules; отдельная база данных не требуется.
- Начальные товары: Keyboard — 15000 и Mouse — 8000. Вкладка «Категории» изначально пустая.

Сведения о категориях хранятся в отдельном массиве JavaScript в памяти сервера. Связи между товарами и категориями не создаются. Даже если названия совпадают, это независимые данные. После перезапуска сервера созданные записи пропадают.

4. Исходная структура проекта

Готовая структура

```
shop-admin/
├── app.js
├── package.json
├── routes/productRoutes.js
├── controllers/productController.js
├── services/productService.js
└── public/
    ├── index.html
    ├── css/style.css
    └── js/api.js, ui.js, app.js
```

Студенты добавляют файлы routes/categoryRoutes.js, controllers/categoryController.js, services/categoryService.js и изменяют клиентские файлы. Никакие файлы для заказов создавать не требуется.

5. Проверка готового проекта

Перейдите в каталог проекта, установите зависимости. Если Node.js и npm уже установлены, повторно устанавливать их не нужно.

Установка зависимостей и запуск

```
npm install
npm start
```

Ожидаемый результат: в терминале появляется http://localhost:3000. Откройте этот адрес в браузере: во вкладке «Товары» присутствуют два товара, форма позволяет добавить третий. Вкладка «Категории» отображает заглушку.

Проверка исходного GET

```
curl -i http://localhost:3000/api/products
```

Критерий перехода: получен HTTP 200 и JSON-массив товаров. Если адрес недоступен, проверьте запуск сервера и порт 3000.

6. Что должно получиться

REST API категорий

Метод	URL	Результат
GET	/api/categories	200 OK и JSON-массив категорий
POST	/api/categories	201 Created и созданная категория
POST (ошибка)	/api/categories	400 Bad Request и JSON с message

Модель категории

```
{
  "id": 1,
  "name": "Accessories"
}
```

Модель категории состоит только из id и name. Идентификатор присваивает сервер, название вводит пользователь. Не добавляйте description, productId, заказы и другие поля.

7. Основное задание: backend категорий

7.1. Создать Service

В файле services/categoryService.js создайте независимый массив categories и счётчик nextId. Напишите функции getAllCategories() и createCategory(data). Первая возвращает массив, вторая формирует объект с новым id, добавляет его в массив и возвращает результат. Используйте productService.js как образец, но не копируйте товары в категории.

7.2. Создать Controller

В controllers/categoryController.js реализуйте getAll(req, res) и create(req, res). GET возвращает категории методом res.json(). POST получает name из req.body, проверяет, что это непустая строка, и возвращает HTTP 201 с новой категорией. Если имя отсутствует или состоит из пробелов, возвратите HTTP 400 и {"message":"Введите название категории"}.

7.3. Создать Router

В routes/categoryRoutes.js создайте Express Router и зарегистрируйте только два обработчика: router.get("/", ...) и router.post("/", ...). Экспортируйте router по умолчанию.

7.4. Подключить Router

В app.js импортируйте categoryRoutes и подключите его через app.use("/api/categories", categoryRoutes). Сохраните действующий Product API. Перезапустите сервер для применения изменений.

7.5. Проверить backend

GET категорий

```
curl -i http://localhost:3000/api/categories
```

Ожидается HTTP 200 и JSON-массив (первоначально допустим пустой []).

POST категории

```
curl -i -X POST http://localhost:3000/api/categories -H "Content-Type: application/json"
-d '{"name":"Accessories"}'
```

Ожидается HTTP 201 и JSON с id и name. Затем повторите GET: новая категория должна присутствовать в массиве.

Отрицательный тест

```
curl -i -X POST http://localhost:3000/api/categories -H "Content-Type: application/json"
-d '{"name":""}'
```

Ожидается HTTP 400 и понятное сообщение. Некорректная категория не должна добавляться. Если команды curl с указанными кавычками неудобны в Windows PowerShell, используйте терминал Bash или отправьте запрос через Postman.

8. Основное задание: frontend категорий

8.1. Создать форму и список

В public/index.html замените заглушку внутри #categories-panel на форму с id="category-form", текстовым input с id="category-name", кнопку «Добавить категорию», абзац #category-message и список ul#categories. Не меняйте форму товаров.

8.2. Добавить API-функции

В public/js/api.js добавьте getCategories() (GET /api/categories) и createCategory(category) (POST /api/categories с Content-Type: application/json и body: JSON.stringify(category)). Используйте уже готовую функцию request(), которая проверяет response.ok и читает JSON.

8.3. Отображение DOM

В public/js/ui.js создайте renderCategories(categories). Функция должна очищать #categories, формировать элемент li для каждой категории и выводить её название через textContent. Для пустого массива покажите «Категорий пока нет». Добавьте вывод сообщения об ошибке или успехе в #category-message.

8.4. Обработка событий

В public/js/app.js подключите новые API- и UI-функции. Напишите async loadCategories(): вызвать getCategories(), затем renderCategories(). Вызывайте её при выборе вкладки «Категории». На submit формы отменяйте стандартную отправку event.preventDefault(), получайте name, вызывайте createCategory({name}), очищайте форму после успеха и повторно загружайте список.

Операции GET и POST должны выполняться без полной перезагрузки страницы. Для HTTP-ошибок используйте try/catch и показывайте текст ошибки пользователю. Не добавляйте PATCH, PUT, DELETE и кнопки редактирования или удаления.

9. Диагностический эксперимент

Введите пустое название категории и попробуйте отправить форму. Браузер может остановить отправку из-за required; для проверки серверной валидации отправьте POST с пустым именем через curl/Postman. Сервер должен ответить 400; число категорий не меняется. Затем отправьте правильное имя и подтвердите ответ 201 и появление новой записи через GET и в интерфейсе.

10. Итоговая проверка

1. Открыть / и убедиться, что вкладка «Товары» по-прежнему показывает товары и добавляет новые.
2. Переключиться во вкладку «Категории»: получить список через GET /api/categories.
3. Добавить категорию «Accessories» через форму: получить ответ POST 201 и обновлённый список.
4. Добавить вторую категорию «Components», убедиться, что отображаются обе.
5. Проверить некорректный POST: вернуть 400 и не создавать запись.
6. Перезагрузить браузер: добавленные категории остаются, пока Node.js сервер работает.
7. Перезапустить Node.js сервер: данные в памяти возвращаются к исходному состоянию.

11. Содержание отчёта

8. Тема, цель и использованная среда.
9. Схема взаимодействия: frontend → fetch() → Express Router → Controller → Service → JSON → DOM.
10. Листинги трёх новых backend-файлов и изменённых фрагментов app.js, index.html, api.js, ui.js, app.js.

11. Скриншот вкладки «Товары» и работающей вкладки «Категории».
12. Результаты GET, POST и ошибки 400 (скриншоты или вывод curl/Postman).
13. Объяснение, почему созданные категории исчезают после перезапуска сервера.
14. Краткий вывод о том, как связаны frontend и backend.

Работа считается выполненной, если студент самостоятельно добавил просмотр и создание категорий через браузер и REST API, не нарушив работу исходного примера товаров. Реализация заказов и других HTTP-методов не обязательна.